INTRODUCTION

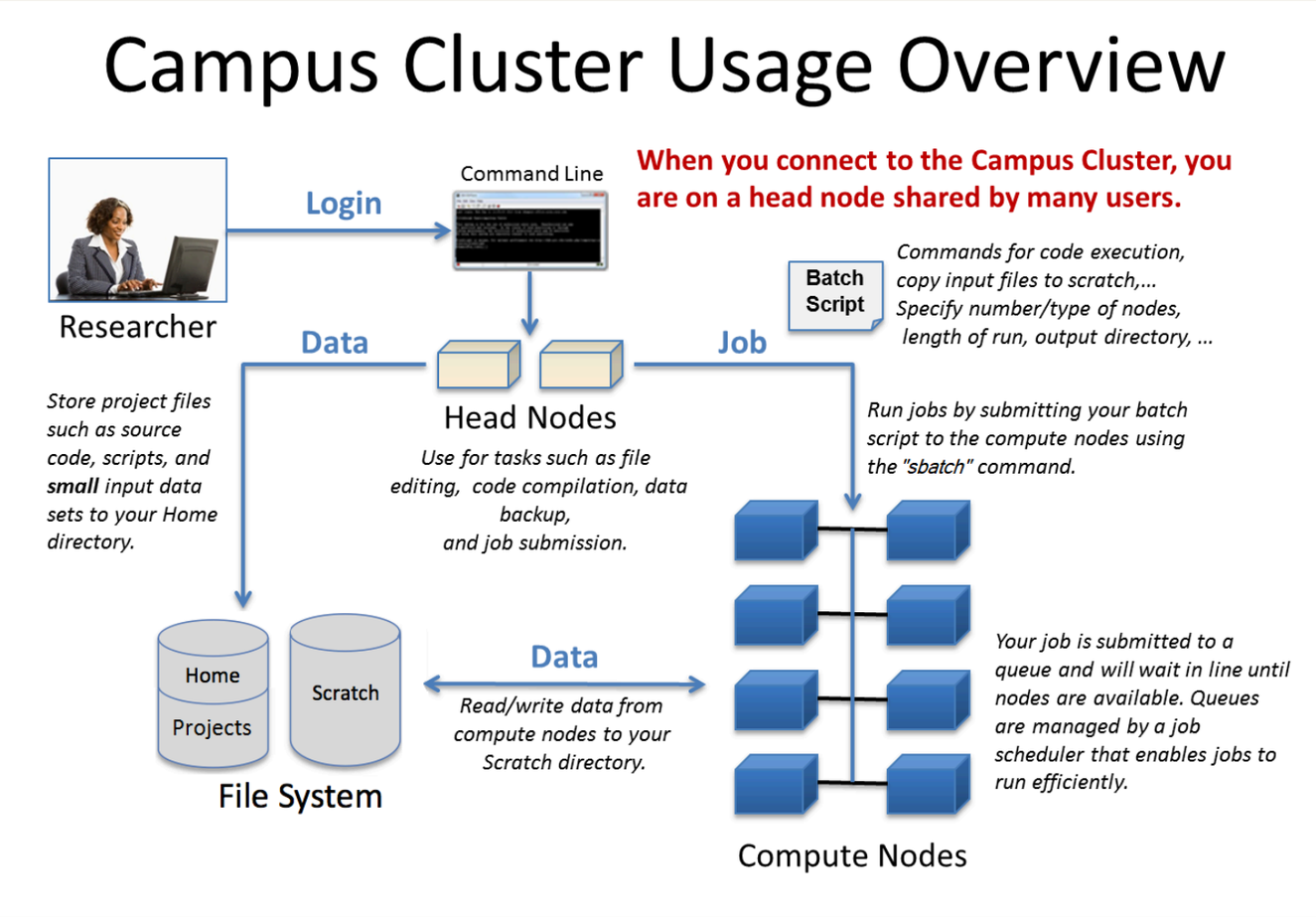
Getting started on the Illinois Campus Cluster

A practical workflow for data analysis



Illinois Computes · Official photograph

Prepare files on login nodes; run jobs on compute nodes



HPC = high-performance computing. A node is one computer. Login: prepare files. Compute: run scheduled analysis.

Request access and confirm your allocation details

Existing lab access

Ask your PI or technical contact which cluster group to join.

A new computing need

Start an Illinois Computes request.

Detail to confirm	Used for
NetID	Your login identity
Slurm account	The group's assigned computing resources
Partition	A queue you may submit to
Project path	A folder where you have permission to save files

Research access request ↗ Illinois Computes request ↗

Before connecting: get account approval, confirm these four details, and set up Duo.

The first SSH login

On your laptop: open Terminal (macOS) or PowerShell (Windows)

```
ssh YOUR_NETID@cc-login.campuscluster.illinois.edu
```

`ssh` starts the connection · before `@`: your user name · after `@`: the cluster address

1 · Run the command

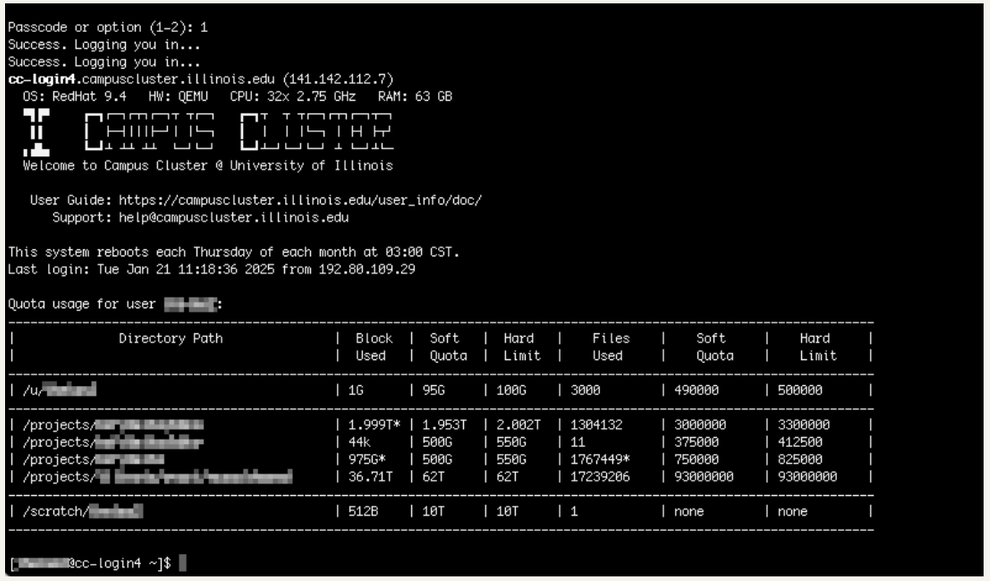
Replace YOUR_NETID, then press Enter.

2 · Password

Type your NetID password and press Enter; typing stays hidden.

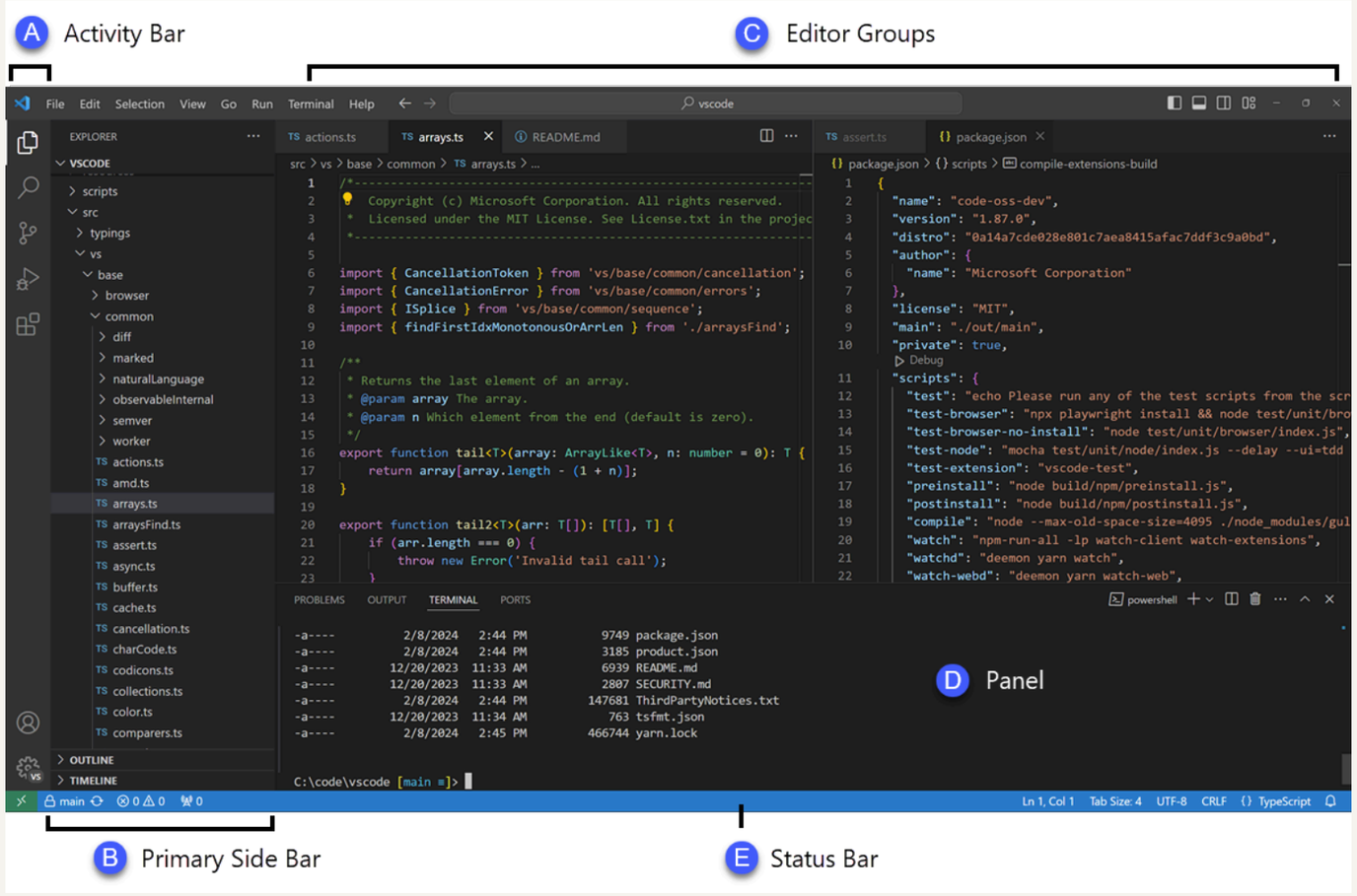
3 · Duo

Follow the prompt and approve the authentication request.



A welcome message and cluster command prompt confirm login. Screenshot dates and quotas are historical.

VS Code is an editor for code and project files



Microsoft example: B = files, C = editor, D = terminal panel.

Visual Studio Code (VS Code) is Microsoft's code editor for working with scripts and project files.

B · Explorer

Browse files and folders in the left sidebar.

C · Editor

Read and edit scripts in the main area.

D · Terminal

Enter commands in the bottom panel.

Files stay on the cluster; terminal commands run there

LOCAL

Your laptop

VS Code window

View and edit

`analysis.py`

Your screen, keyboard, and mouse

Remote - SSH



View · edit · save

REMOTE

The cluster

Project files

`demo/analysis.py`

Save updates this file.

Remote terminal

Commands run on the cluster.

In the SSH window, saves update cluster files and terminal commands run on the connected host.

Install VS Code and the Remote - SSH extension

First: download and open VS Code on your laptop.

1 Open Extensions

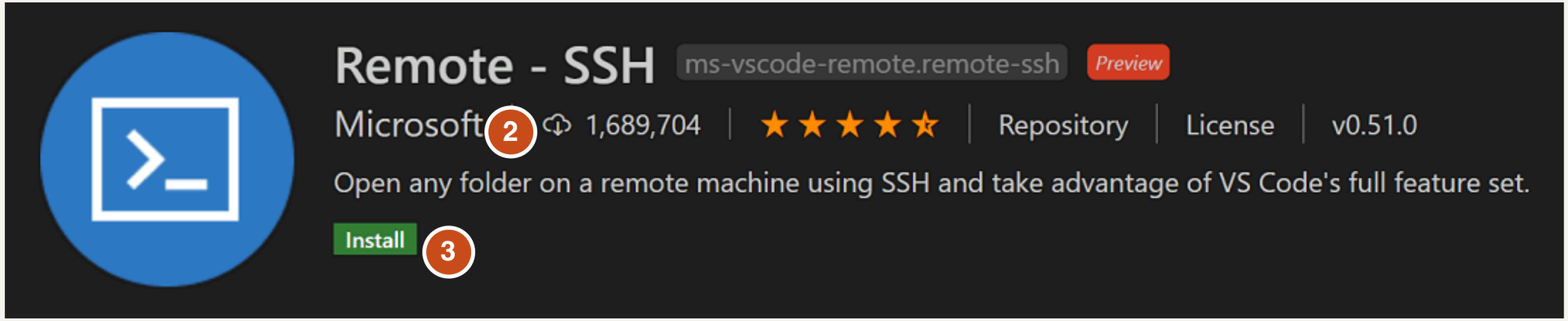
Left sidebar, or Cmd/Ctrl+Shift+X

2 Search Remote - SSH

Choose the Microsoft extension

3 Click Install

Wait for installation to finish



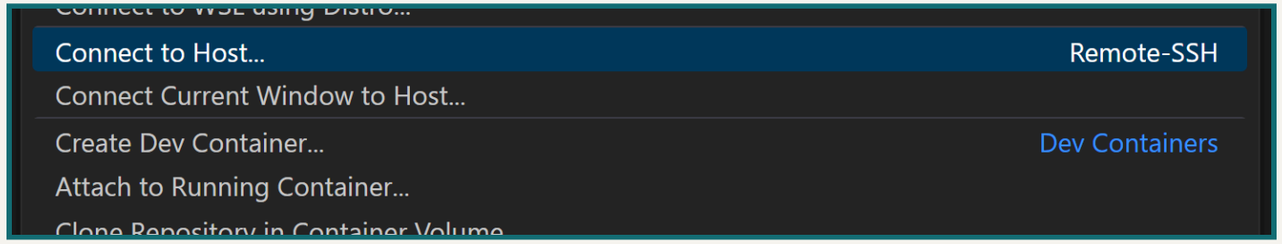
Microsoft's example screenshot uses an older version; install the current extension.

> The editor and Remote - SSH extension are installed on your laptop.

Connecting VS Code with Remote - SSH

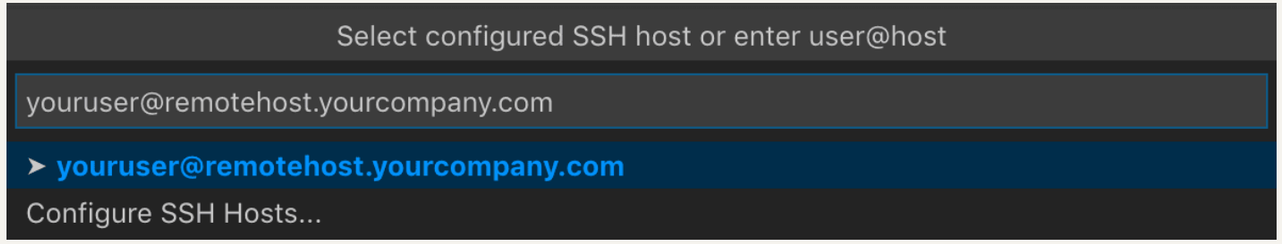
1 Choose Connect to Host...

Cmd+Shift+P (Mac) · Ctrl+Shift+P (Windows/Linux)
Search: Remote-SSH: Connect to Host



Microsoft menu example · focused view

2 Enter your login address



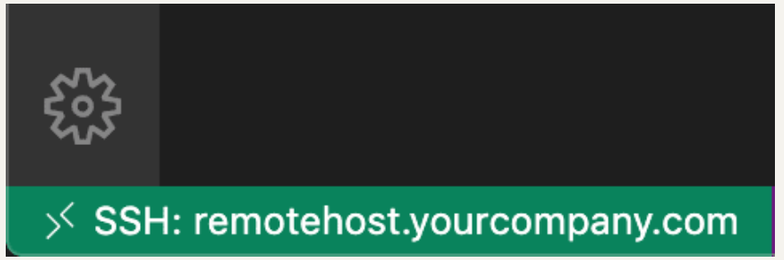
Use this address:

`YOUR_NETID@`
`cc-login.campuscluster.illinois.edu`

Password + Duo
Choose **Linux** if asked.

3 Check the SSH indicator

Lower-left corner of the new window



Microsoft's example host is shown here.
Your indicator should name the cluster.

Choose home, project, or scratch storage

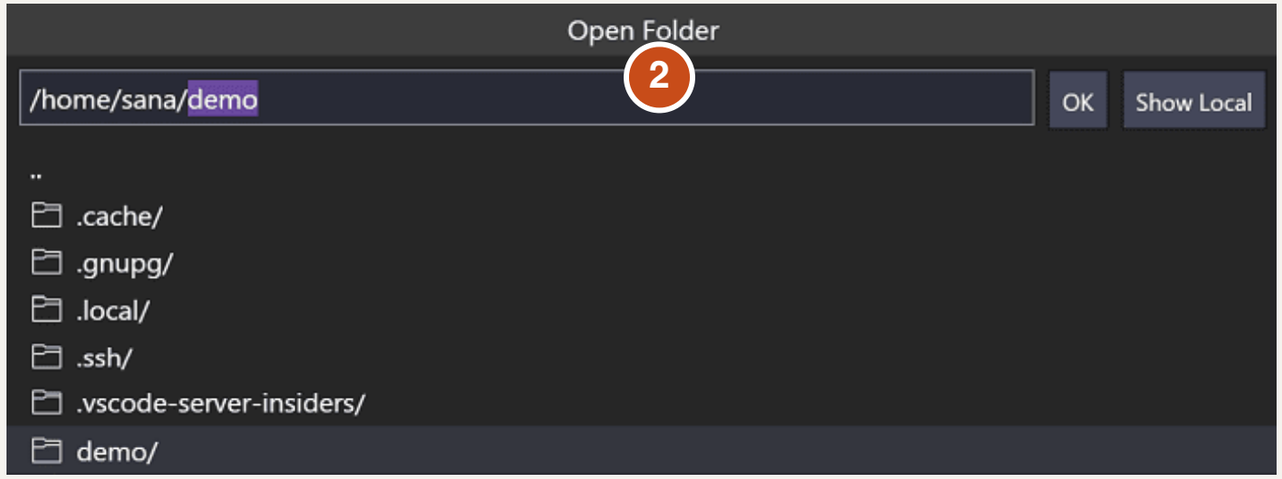
Location	Use it for	Limits and actions
/u/YOUR_NETID	Settings and small personal files	Very limited quota: 100 GiB (soft limit). Keep large datasets elsewhere.
/projects/YOUR_GROUP	Group data, software, and retained outputs	Shared group quota. Keep backups: no snapshots are kept for file recovery.
/scratch/YOUR_NETID	Temporary working data	Temporary only: files typically deleted after 30 days without access, sometimes sooner.

> Use your group's full project path; `/projects/YOUR_GROUP` is a placeholder.

Open the remote project location

1 File → Open Folder

In the connected VS Code window



Microsoft's sample path is /home/sana/demo.
Replace it with your own approved project path.

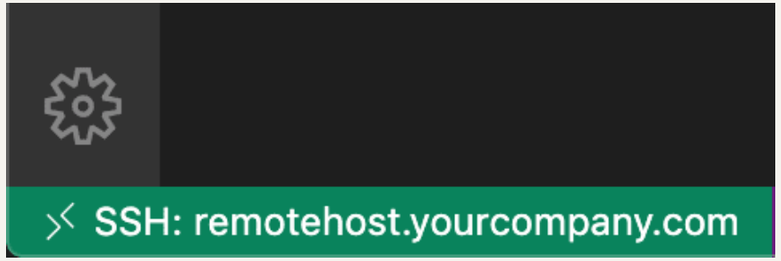
2 Enter the destination, then OK

Use your group's full path, for example:

```
/projects/YOUR_GROUP
```

3 Confirm this window is remote

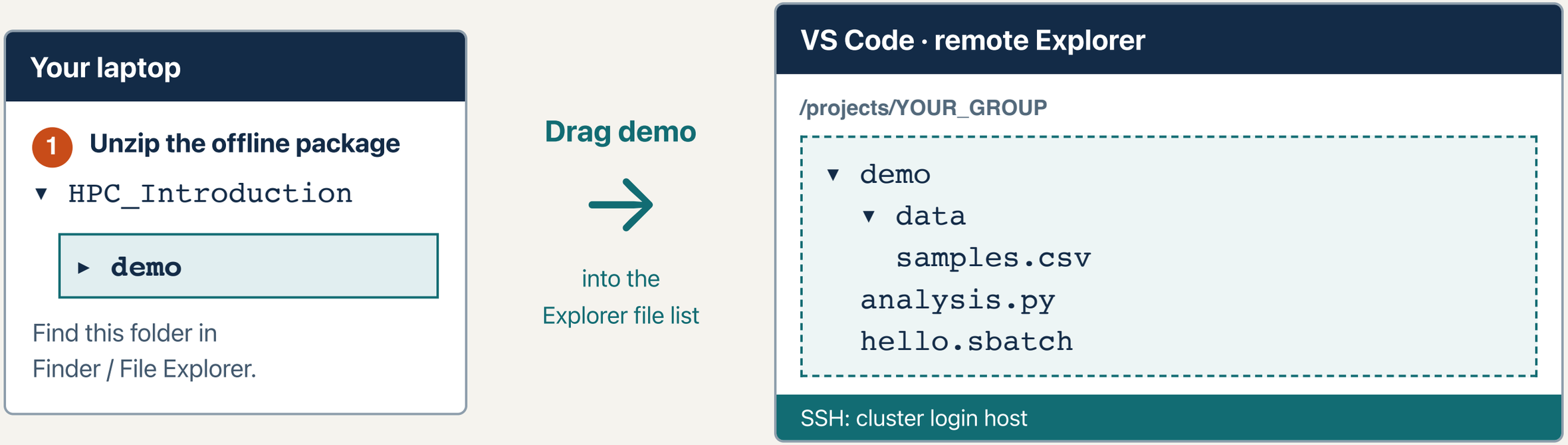
The lower-left SSH indicator names the cluster.



> Explorer now shows the remote destination.
> The demo folder is still on your laptop.

Upload the demo folder with VS Code

Illustration · small project upload



2 **After upload: open demo**

File → Open Folder → /projects/YOUR_GROUP/demo

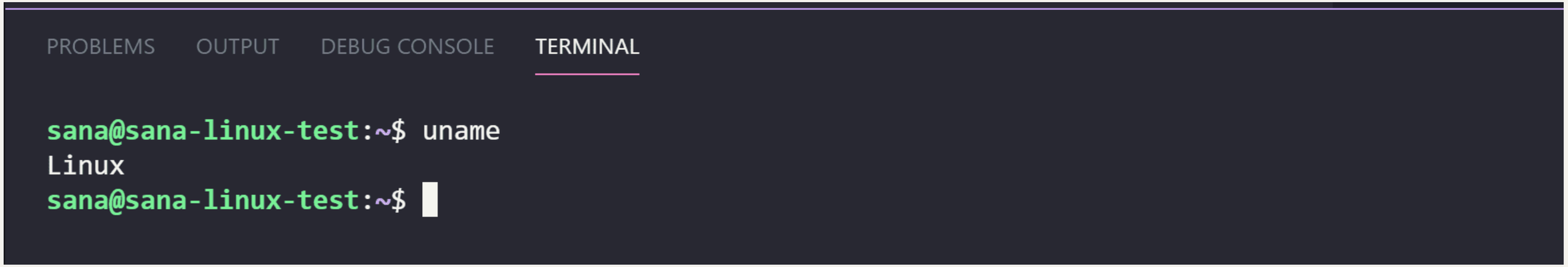
3 **Open data/samples.csv**

Check the header and three sample rows.

Open a terminal in the same project

1 Terminal → New Terminal

In your remote VS Code window



Microsoft's remote terminal example. Run our three checks below in your own project.

`hostname`	`cd /projects/YOUR_GROUP/demo`	`pwd`
Which computer?	Move into your uploaded project.	Which folder?
Expect a login node.	Use your group's actual path.	Check the full project path.

> Run one line, press Enter, wait for the prompt, then continue.

Read the program, options, and path in a command

```
head -n 5 data/samples.csv
```

Program	head displays the beginning of a text file.
Option + value	-n 5 asks for the first five lines.
File path	data/samples.csv is the path to the file to read.

> Quotes keep a path together: `head -n 5 "data/samples.csv"`. They are required for paths containing spaces.

Navigate the uploaded project

Transcript: type commands after $; other lines are output

```
$ pwd
/projects/YOUR_GROUP/demo
$ cd data
$ pwd
/projects/YOUR_GROUP/demo/data
$ cd ..
$ pwd
/projects/YOUR_GROUP/demo
$ cd ~
$ pwd
/u/YOUR_NETID
$ cd -
/projects/YOUR_GROUP/demo
$ pwd
/projects/YOUR_GROUP/demo
```

Command	Meaning
pwd	Print the current folder
cd data	Enter the data subfolder
cd ..	Go up one folder
cd ~	Go to your home folder
cd -	Return to the previous folder

`cd -` also prints its destination. The next `pwd` confirms it.

`.` current folder · `..` parent · `/projects/…` absolute path · `data/…` relative path

List files and inspect their details

Command	What changes
ls	List visible file and folder names
ls -a	Include hidden names beginning with a dot
ls -l	Show details such as size and date
ls -lh	Use readable sizes, such as K or M
ls -lah	Show hidden names, details, and readable sizes

Illustrative listing with a larger file; total lines omitted

```
$ ls data
counts.tsv  samples.csv
$ ls -a data
.  ..  counts.tsv  samples.csv
$ ls -l data
-rw-r--r-- 1 user group 25165824 Oct 27 10:00 counts.tsv
-rw-r--r-- 1 user group       58 Oct 27 10:00 samples.csv
$ ls -lh data
-rw-r--r-- 1 user group  24M Oct 27 10:00 counts.tsv
-rw-r--r-- 1 user group   58 Oct 27 10:00 samples.csv
$ ls -lah data
drwxr-xr-x 2 user group 4.0K Oct 27 10:00 .
drwxr-xr-x 3 user group 4.0K Oct 27 10:00 ..
-rw-r--r-- 1 user group  24M Oct 27 10:00 counts.tsv
-rw-r--r-- 1 user group   58 Oct 27 10:00 samples.csv
```

With `-h`: **25165824 bytes → 24M**. `counts.tsv` is illustrative; the demo still uses `samples.csv`.

Creating, copying, moving, and deleting

Command	Result
`mkdir results`	Create one new folder
`mkdir -p results/qc`	Create missing folders along the path; accept existing folders
`cp -i notes.txt notes_backup.txt`	Copy a file; ask before overwriting
`cp -r data data_backup`	Copy the folder and all its subfolders to a new location
`mv -i notes_backup.txt notes_old.txt`	Rename or move; ask before overwriting
`rm -i notes_old.txt`	Ask before deleting this file

`cp` and `mv`: source first, destination second. Shell deletion normally has no Trash recovery.

Inspecting a small text file

Command	Use
`cat notes.txt`	Print the entire small file
`head -n 5 data/samples.csv`	Print up to 5 lines from the start
`less notes.txt`	Browse text; press q to leave

Output of head for the included sample table

```
sample_id,reads
sample_A,1000
sample_B,1500
sample_C,1200
```

> Preview a few rows before running the analysis. Avoid printing a huge file with `cat`.

Find a file, then find text inside it

Search filenames within data and its subfolders

```
find data -type f -name "*.csv"
```

-type f / -name	Keep files only / match their names against the pattern.
"*.csv"	The * matches any text before .csv.

Search inside this file; -n includes line numbers

```
grep -n "sample_B" data/samples.csv
```

Expected grep output: line 3 contains the match

```
3:sample_B,1500
```

Count lines and check disk usage

Run this command

```
wc -l data/samples.csv
```

wc -l

Count lines in this text file.

Expected output; do not type this line

```
4 data/samples.csv
```

4 lines

One header plus three sample rows.

Check space used by the input folder

```
du -sh data
```

`-s` gives one summary total · `-h` uses readable units such as K or M

> `wc -l` includes header lines. `du` shows space used; `quota` shows usage and limits.

Help and everyday terminal shortcuts

Command / key	What it does
`man ls`	Open the manual; press q to leave
`head --help`	Show the Linux command's built-in help
Tab	Complete a file or folder name
↑ / `history`	Recall previous commands / list shell history
Ctrl+C	Interrupt the command currently running in this terminal
`clear` / `exit`	Clear the display / leave the current shell

Keep this login shell open for job submission; exit closes it.

Slurm runs work on allocated resources

LOGIN NODE

Prepare and submit

`hello.sbatch`

Your terminal stays here.

`sbatch` →

Slurm

Wait for resources

The scheduler

Allocate →

COMPUTE NODE

Run the job

`hello.sbatch`

The program runs here.

↕ ↕

Same shared `demo/` folder Input · scripts · output

Batch: submit a script. **Interactive:** request a compute shell with `srun`.

> Submit on the login node; the job runs on an allocated compute node.

Read a Slurm batch script line by line

```
01 #!/bin/bash
02 #SBATCH --job-name=hello
03 #SBATCH --account=YOUR_ACCOUNT
04 #SBATCH --partition=YOUR_PARTITION
05 #SBATCH --nodes=1
06 #SBATCH --ntasks=1
07 #SBATCH --cpus-per-task=1
08 #SBATCH --mem=1G
09 #SBATCH --time=00:05:00
10 echo "My first HPC job"
```

01 Use Bash to read the commands.

02 Give the job a recognizable name.

03 Use the Slurm account assigned to your group.

04 Use a partition that this account can access.

05 Request one compute node.

06 Request one task for this program.

07 Reserve one CPU for that task.

08 Request 1 GiB of RAM per node.

09 Set a five-minute running-time limit.

10 Print a message into the job output.

Edit these lines inside hello.sbatch in VS Code, then save. Submit it from the terminal after reviewing the resource settings.

Set nodes, tasks, CPUs, memory, and time for a job

Example resource request: count the node, the tasks, then the CPUs.

`--nodes=1`

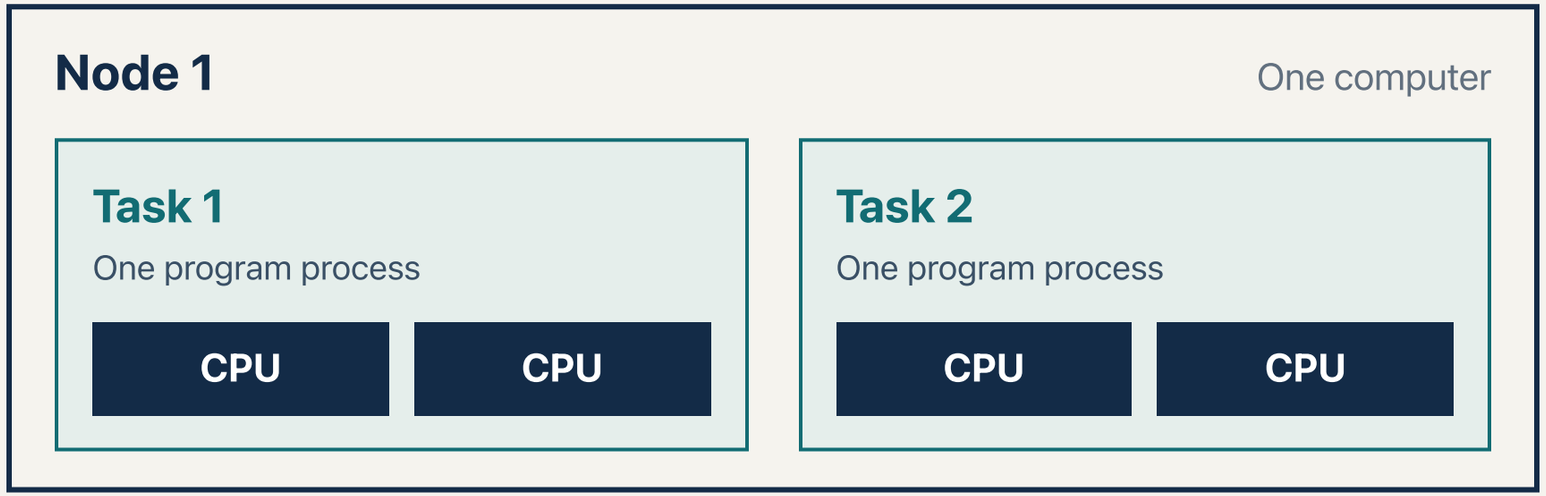
1 computer

`--ntasks=2`

2 tasks in the whole job

`--cpus-per-task=2`

2 CPUs reserved for each task



> **2 tasks × 2 CPUs/task = 4 CPUs requested**
>
> ntasks is already the total; do not multiply by nodes again.

Reserving resources does not launch the tasks or make the program parallel.

Keep hello.sbatch at: 1 node · 1 task · 1 CPU/task

`--mem=1G` 1 GiB of RAM per node

`--time=00:05:00` Runtime limit: hours:minutes:seconds

Submit the first job: hello.sbatch

In the remote demo folder, after saving the file

```
pwd
ls hello.sbatch
sbatch hello.sbatch
```

Example response; record the ID returned by your submission

```
Submitted batch job 123456
```

> This first job prints a message. It does not run analysis.py yet.

Checking the queue and stopping a job

-u selects your user; run squeue again to refresh

```
squeue -u YOUR_NETID
```

Illustration: job 123456

PD

Pending

Queued; check the reason

→

R

Running

Using allocated resources

→

No longer listed

Read the output

`slurm-123456.out`

> Leaving the queue does not prove success.

Optional: cancel an unwanted job. Let hello finish here.

```
scancel 123456
```

After hello leaves the queue: `cat slurm-123456.out` → **My first HPC job**. Use your job ID.

Output files and email notifications

In analysis.sbatch, before executable commands

```
#SBATCH --output=analysis-%j.out
#SBATCH --error=analysis-%j.err
#SBATCH --mail-type=END,FAIL
#SBATCH --mail-user=YOUR_NETID@illinois.edu
```

Illustration: analysis job 123457. Slurm fills in `%j`.

Program output Ordinary messages	→	`analysis-123457.out`
Program diagnostics Errors and other messages	→	`analysis-123457.err`
Slurm events `END` or `FAIL`	→	Email to `YOUR_NETID@illinois.edu`

An `.err` file can also contain progress messages. Read its contents.

`%j` becomes the job ID. Use your email address, or remove both mail lines.

When the analysis needs a GPU

Optional GPU job: use your group’s GPU partition and GPU type

```
#SBATCH --partition=YOUR_GPU_PARTITION
#SBATCH --gres=gpu:GPU_TYPE:1
```

Inside an allocated NVIDIA GPU job

```
nvidia-smi
```

--gres	Request GPUs with gpu:TYPE:COUNT; count is per node.
GPU_TYPE / final 1	The configured GPU type / one GPU per allocated node.

> The tiny demo needs no GPU. Add this only for software that supports and uses one.

When a particular node is required

Optional: use a compute-node name your account can access

```
#SBATCH --nodes=1
#SBATCH --nodelist=NODE_NAME
```

sinfo: inspect resources; -p selects the partition; -N lists nodes

```
sinfo -p YOUR_PARTITION -N
```

--nodes=1	How many nodes you request
--nodelist	Which named node you request

> Normally let Slurm choose. Name a compute node only when your group requires that machine.

Open an interactive compute shell with srun

From the login shell in demo: paste this whole block as one command

```
srun --account=YOUR_ACCOUNT --partition=YOUR_PARTITION \
  --nodes=1 --ntasks=1 --cpus-per-task=2 \
  --mem=4G --time=00:15:00 --pty bash
```

srun	Start a program on allocated compute resources.
--pty bash	Open an interactive terminal running the Bash shell.
After the prompt returns	hostname should show a compute node; pwd should show the project folder.

A trailing `\` continues a command onto the next line; nothing follows it on that line.

Make Python available in the compute shell

Inside the allocation, in demo; use a version shown as available

```
module avail python
module load python/3.11.11
module list
```

Command	Meaning
module avail python	List cluster modules matching Python
module load python/3.11.11	Select this installed Python version for this shell
module list	Confirm which cluster modules are loaded

> analysis.py needs pandas. Next, create .venv in demo and install pandas there.

Create a Python environment and install pandas

Continue in the same compute shell and remote demo folder

```
python -m venv .venv
source .venv/bin/activate
python -m pip install pandas
ls -a
```

Create .venv	-m runs Python’s venv utility; .venv is the new folder.
Activate	source makes this shell use .venv/bin/python.
Install pandas	-m pip runs the package installer for the selected Python.

Wait for pip to finish and the prompt to return. Then check the import on the next slide.

Check that pandas imports and save package versions

-c executes the quoted Python code immediately

```
python -c "import pandas; print(pandas.__version__)"
```

import pandas	Check that this Python can load the package.
Print version	print(pandas.__version__) displays the installed version; expect a version number.

freeze lists installed package versions; > writes them into this file

```
python -m pip freeze > requirements.txt
```

> Continue when a version prints without an error. Keep this compute shell open.

The environment stays; activation is per shell

Install packages

```
python -m pip install pandas
```

After creating and activating .venv

Write files →

Shared project storage

`demo/.venv/`

Python + installed packages

These files remain.

Activate in each new shell or job

```
source .venv/bin/activate
```

Current compute shell

```
python → .venv/bin/python
```

Same existing

.venv

No reinstall

A later batch job

```
python → .venv/bin/python
```

> After setup, `exit` returns to the login shell; .venv stays in project storage.

Create a Conda environment from a supplied file

Alternative route when the project supplies environment.yml

```
module avail miniconda3
module load YOUR_CONDA_MODULE
conda env create -f environment.yml
conda run -n analysis python -c "import pandas; print(pandas.__version__)"
```

environment.yml	Records the environment name and requested packages
-f environment.yml	Read the environment specification from this file.
-n analysis	Use the environment named analysis (from name: analysis in the file).

Choose this instead of .venv. In a new compute shell, expect a pandas version with no error.

Prepare a Python batch job with its environment

1 · Beginning of analysis.sbatch

```
#!/bin/bash
#SBATCH --job-name=analysis
#SBATCH --account=YOUR_ACCOUNT
#SBATCH --partition=YOUR_PARTITION
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:05:00
```

2 · Continue in the same file, after the left column

```
#SBATCH --output=analysis-%j.out
#SBATCH --error=analysis-%j.err

set -e
module load python/3.11.11
source .venv/bin/activate
python analysis.py
```

> set -e stops on setup failure. Set your account and partition, then save analysis.sbatch.

Submit the analysis job

In the login shell, from the demo folder

```
pwd
ls data/samples.csv analysis.py analysis.sbatch
sbatch analysis.sbatch
```

Illustrative analysis job ID; hello used 123456

```
Submitted batch job 123457
```

Check whether this new job is pending or running

```
squeue -u YOUR_NETID
```

> Use the returned ID in analysis-JOBID.out and analysis-JOBID.err.

Watch job output and search the error log

After the job starts and the log exists

```
tail -n 20 analysis-123457.out
tail -f analysis-123457.out
```

`-n 20`: last 20 lines · `-f`: follow new output

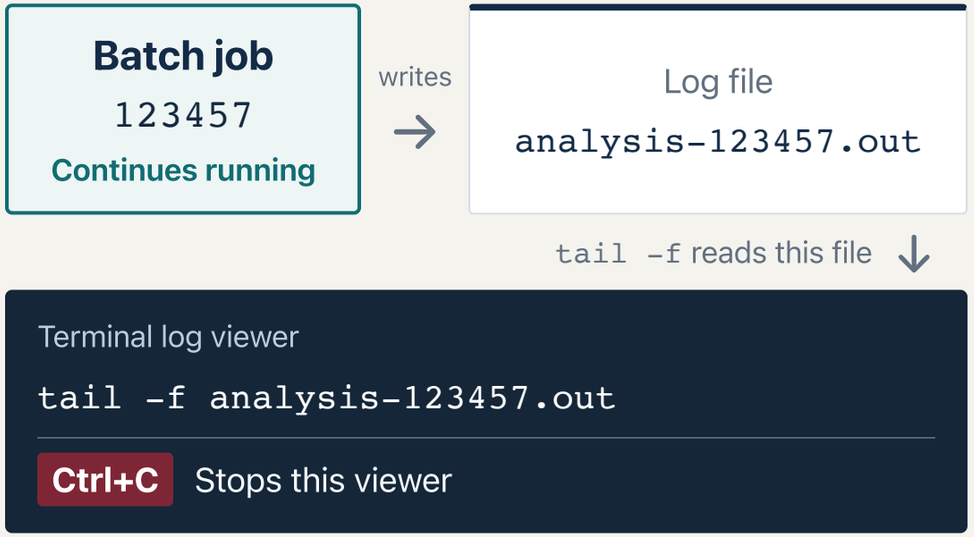
After Ctrl+C stops the viewer

```
grep -in "error" analysis-123457.err
```

`-i`: ignore case · `-n`: show line numbers

> To cancel the analysis job itself: `scancel 123457`.

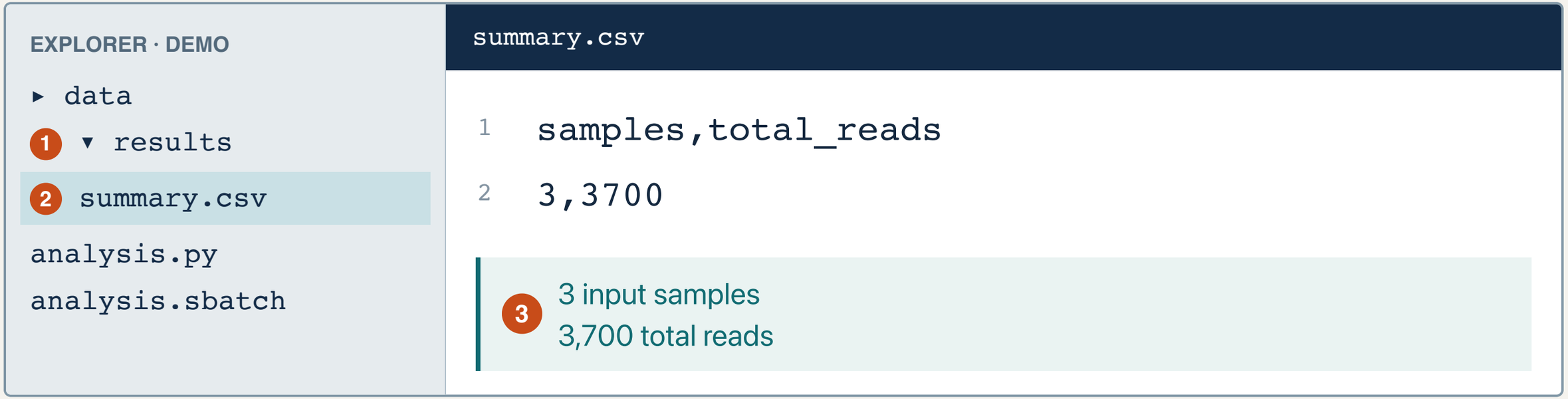
Illustration: a job and its log viewer



The job and log viewer are separate processes.

Open the result in VS Code

Illustration · expected result from the supplied demo



Read it here. No download needed.
For a local copy: right-click the remote file → **Download…**

Other viewers: logs and images open directly;
Rainbow CSV colors columns; PDF/Jupyter extensions open those files.

Microsoft · Remote - SSH · Rainbow CSV · Extension documentation · vscode-pdf · Extension documentation · Microsoft · Jupyter extension · Microsoft · File import and download implementation

Run an R script as a Slurm batch job

R commands below the #SBATCH resource settings

```
module load R
Rscript analysis.R
```

Copy analysis.R and analysis_R.sbatch into the project; set the account and partition

```
sbatch analysis_R.sbatch
```

Same input	data/samples.csv
R output	results/summary_R.csv

> This example uses base R; no additional package installation is needed.

Common errors and the first useful check

What you see	Check first
No such file or directory	Run pwd; check the path and filename capitalization.
Command not found	Check spelling; load the module or activate the environment.
No module named… / no package called…	Select the intended Python/R environment; install the missing package there.
OUT_OF_MEMORY / TIMEOUT	Compare --mem / --time with the tool’s documented requirements.
Invalid account / partition	Use your approved --account and matching --partition.
Permission denied / quota exceeded	Run quota; confirm the group gave you write access to that folder.

Documentation and help

ICC · Accessing the System

ICC · Storage and Data

Missing Semester · Introduction to the Shell (2026)

Microsoft · Remote - SSH

Python Packaging · pip and venv

Conda · Managing environments

ICC · Running Jobs

ICC · Software and Modules

Missing Semester · Command-line Environment (2026)

SchedMD · sbatch

Bioconductor · Installation

Reveal.js · Vertical slides

ICC support: **help@campuscluster.illinois.edu**
Full references and image credits are in SOURCES.md.